CPE333 Operating Systems (1/2026)
Problem Session 2: Process Creation & Pipes

1. Name: Thanaboon Tikaew **ID:** 67070501021
2. Name: Chanon Lhumsa-ard **ID:** 67070501059
3. Name: Theeraphat Jaingam **ID:** 67070501063
4. Name: Siriwan Yindeephot **ID:** 67070501075

Environment: Ubuntu 24.04 on WSL2, gcc 15.2.0, 8 CPUs

1 fork() examples (P1)

Two programs were written following the `fork()` example from the lecture slide. Program 1.1 (`p1a.c`) is the plain `fork()` example. Program 1.2 (`p1b.c`) is the same program modified with the `wait()` function in the parent.

1.1 helloworld (p1a.c)

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

int main(int argc, char *argv[])
{
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) {
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) {
        printf("hello, I am child (pid:%d)\n", (int) getpid());
    } else {
        printf("hello, I am parent of %d (pid:%d)\n", rc, (int) getpid());
    }
    return 0;
}
```

How it works: `fork()` creates a child process that is an almost exact copy of the parent. In the child the return value `rc` is 0; in the parent `rc` is the child's PID. Each branch prints its own message; both processes continue running concurrently after `fork()`.

1.2 modified with wait() (p1b.c)

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

int main(int argc, char *argv[])
{
    printf("hello world (pid:%d)\n", (int) getpid());
    int rc = fork();
    if (rc < 0) {
        fprintf(stderr, "fork failed\n");
```

```
12          exit(1);
13      } else if (rc == 0) {
14          printf("hello, I am child (pid:%d)\n", (int) getpid());
15      } else {
16          int wc = wait(NULL);
17          printf("hello, I am parent of %d (wc:%d) (pid:%d)\n", rc, wc, (int)
        getpid());
18      }
19      return 0;
20  }
```

How it works: the parent calls `wait(NULL)`, which blocks (sleeps) until the child terminates and then reaps the child's exit status. Only after that does the parent print its message and exit.

Results (three runs of each)

```
=== P1a: fork without wait ===
--- run 1 ---
hello world (pid:1237)
hello, I am parent of 1238 (pid:1237)
hello, I am child (pid:1238)
--- run 2 ---
hello world (pid:1239)
hello, I am parent of 1240 (pid:1239)
hello, I am child (pid:1240)
--- run 3 ---
hello world (pid:1241)
hello, I am parent of 1242 (pid:1241)
hello, I am child (pid:1242)

=== P1b: fork with wait ===
--- run 1 ---
hello world (pid:1243)
hello, I am child (pid:1244)
hello, I am parent of 1244 (wc:1244) (pid:1243)
--- run 2 ---
hello world (pid:1245)
hello, I am child (pid:1246)
hello, I am parent of 1246 (wc:1246) (pid:1245)
```

Discussion

- **Without `wait()` (p1a):** Both parent and child processes execute concurrently after `fork()`. The output order is governed by the OS CPU scheduler and is therefore **non-deterministic** – either process may print first, and the order can vary between runs.
- **With `wait()` (p1b):** The parent blocks inside `wait()` until the child terminates. This synchronisation enforces a **deterministic** execution order in which the child's output always precedes the parent's, eliminating any race condition on output ordering.

2 sleep() + ps: zombie and orphan scenarios (P2)

2.1 Child dies while parent is still running (p2a.c)

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

/* Scenario 2.1: child dies first, parent keeps running (sleeps).
   Child becomes a zombie (<defunct>) until parent exits/reaps. */
int main(int argc, char *argv[])
{
    int rc = fork();
    if (rc < 0) {
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) {
        printf("child (pid:%d) exiting now\n", (int) getpid());
        exit(0);
    } else {
        printf("parent (pid:%d) of child %d sleeping 20s...\n", (int) getpid(),
     rc);
        printf("while parent sleeps, run: ps -o pid,ppid,stat,cmd -e | grep p2a
    \n");
        sleep(20);
        printf("parent (pid:%d) done sleeping, exiting (child was reaped)\n", (
    int) getpid());
    }
    return 0;
}
```

2.2 Parent dies while child is still running (p2b.c)

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>

/* Scenario 2.2: parent dies first, child keeps running (sleeps).
   Child becomes an orphan, reparented to init/systemd. */
int main(int argc, char *argv[])
{
    int rc = fork();
    if (rc < 0) {
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) {
        printf("child (pid:%d) sleeping 20s, parent will die first...\n", (int)
     getpid());
        printf("while child sleeps, run: ps -o pid,ppid,stat,cmd -e | grep p2b\
    n");
        sleep(20);
        printf("child (pid:%d) done sleeping, exiting\n", (int) getpid());
    } else {
        printf("parent (pid:%d) of child %d exiting now\n", (int) getpid(), rc)
    ;
        exit(0);
    }
    return 0;
}
```

Results

```
=== P2a: child dies first, parent sleeps 20 s ===
child (pid:8873) exiting now
 ps while the parent sleeps:
  8871    8865 S+   ./p2a            <- parent still alive (S = sleeping)
  8873    8871 Z+   [p2a] <defunct>  <- child DEAD but kept as a ZOMBIE;
                                        PPID is still its parent (8871)
  8881    8865 S+   grep p2a         <- grep matched itself (ignore)
parent (pid:8871) of child 8873 sleeping 20s...
while parent sleeps, run: ps -o pid,ppid,stat,cmd -e | grep p2a
parent (pid:8871) done sleeping, exiting (child was reaped)

=== P2b: parent dies first, child sleeps 20 s ===
parent (pid:8930) of child 8932 exiting now
 ps while the child sleeps:
  8932    8851 S+   ./p2b            <- only the CHILD remains; its PPID
                                        is no longer 8930 (dead parent)
                                        but 8851 (the session shell acting
                                        as subreaper; on a standard Linux
                                        system: init/systemd, PID 1)
  8940    8865 S+   grep p2b         <- grep matched itself (ignore)
child (pid:8932) done sleeping, exiting
 after 20 s the child finishes and exits normally; no <defunct> entry.
```

Note: on Ubuntu 24.04 (procps 4.x) combining `-f` with an explicit `-o` format pollutes the CMD column with environment variables, so `ps -o pid,ppid,stat,cmd -e` was used instead of `ps -ef`; the columns shown are the same ones `-ef` would give.

Discussion

In Program 2.1 (`p2a`), the child dies first; its `PPID` remains the parent's PID, but its state becomes `Z` (**zombie**/`<defunct>`) because the parent has not called `wait()` to collect the exit status. In Program 2.2 (`p2b`), the parent dies first; the child's `PPID` changes to `init`/`systemd` (reparenting), and the child runs normally as an **orphan** with no zombie produced.

A `<defunct>` process is one that has terminated but whose exit status has not been reaped by its parent via `wait()`. It consumes no CPU or memory, but occupies a PID slot. If zombies accumulate, they can exhaust the PID table. The only remedy is for the parent to call `wait()`, or for the parent to exit so `init` reaps the zombie automatically.

3 How many processes can we fork? (P3)

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <sys/wait.h>

/* Recursive fork counter.
   Parent waits for child, child forks a child, ... until fork()
   fails. Every process counts its own depth (the number of successful
   fork() calls on its path). The deepest process -- the one whose
   fork() fails -- writes the total to /tmp/forkcount.txt and reports
   it. Parents then exit one by one (cascade), and main() reads the
   file and prints the final total.
   CAUTION: run under a controlled process limit (e.g. `ulimit -u`)
   so the chain cannot grow without bound. */

static int depth = 0;

void fork_recursive(void)
{
    int rc = fork();
    if (rc < 0) {
        /* cannot fork anymore: report and record the total */
        FILE *f = fopen("/tmp/forkcount.txt", "w");
        if (f) {
            fprintf(f, "%d\n", depth);
            fclose(f);
        }
        printf("fork() #%d failed at pid %d: cannot fork anymore\n",
               depth, (int) getpid());
        printf("TOTAL successful fork() calls: %d\n", depth);
        exit(0);
    } else if (rc == 0) {
        depth++;            /* this fork() succeeded */
        fork_recursive();   /* child keeps going deeper */
        exit(0);            /* never reached */
    } else {
        wait(NULL);         /* wait for the whole subtree */
        return;             /* only the ROOT unwinds back to main() */
    }
}

int main(void)
{
    printf("fork counter started (pid:%d)\n", (int) getpid());
    fflush(stdout); /* children inherit the buffer: flush first */
    fork_recursive();

    int total = -1;
    FILE *f = fopen("/tmp/forkcount.txt", "r");
    if (f) {
        fscanf(f, "%d", &total);
        fclose(f);
    }
    printf("FINAL total successful fork() calls: %d\n", total);
    return 0;
}
```

How it works: `fork_recursive()` forks a child; the parent waits, while the child increments its own depth counter and recurses, so only one new process is created per level (a chain, not a bomb). The deepest process is the one whose `fork()` fails; it writes the total number of

successful `fork()` calls to `/tmp/forkcount.txt` and reports it. The parents then exit one by one and `main()` reads the file and prints the final total. Because the chain can grow to thousands of processes, the experiment was run under a controlled process limit (`ulimit -u 1000`) so that `fork()` fails quickly with `EAGAIN` and the program terminates by itself (the assignment warns that an uncontrolled fork loop can crash the system).

Results (per-user process limit ulimit -u = 1000)

```
=== P3a: fork until failure, no extra load ===
fork counter started (pid:4958)
fork() #954 failed at pid 1459: cannot fork anymore
TOTAL successful fork() calls: 954
FINAL total successful fork() calls: 954

=== P3b: with 10 background 'stay-running' programs (sleep 300) ===
$ for i in $(seq 1 10); do sleep 300 & done     <- 10 jobs (PIDs 1501-1510)
$ jobs                                          <- all 10 Running
fork counter started (pid:1533)
fork() #944 failed at pid 2490: cannot fork anymore
TOTAL successful fork() calls: 944
FINAL total successful fork() calls: 944

=== P3c: after stopping the background programs ===
$ kill $(jobs -p)                               <- [1]..[10] Terminated
fork counter started (pid:2527)
fork() #954 failed at pid 3485: cannot fork anymore
TOTAL successful fork() calls: 954
FINAL total successful fork() calls: 954
```

Note: the absolute count varies between sessions (e.g. 964 right after a fresh WSL boot, 954 here) because other running tasks share the same budget – only the $-10/+10$ pattern of P3b/P3c is invariant.

System limits observed

```
per-user max processes (ulimit -u) : 31515
kernel threads-max                 : 63031
kernel pid_max                     : 4194304
```

Discussion

The maximum number of processes a user can fork is bounded by the per-user process limit (`ulimit -u`) and available memory. With `ulimit -u` set to 1000, the program achieved 954 successful forks; the remaining slots were occupied by existing system tasks (shells, daemons, threads). In P3b, adding 10 background processes reduced the fork count by exactly 10, and killing them in P3c restored it, confirming that each running process consumes exactly one slot from the shared budget. The absolute count may vary between sessions depending on how many other tasks are running, but the $-10/+10$ pattern is always consistent.

4 Pipe between parent and child (P4)

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <sys/wait.h>

#define BUF_SIZE 128

int main(int argc, char *argv[])
{
    int fd[2];
    if (pipe(fd) < 0) {
        fprintf(stderr, "pipe failed\n");
        exit(1);
    }
    int rc = fork();
    if (rc < 0) {
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) {
        /* child = sender */
        close(fd[0]); /* close read-end */
        char msg[BUF_SIZE];
        snprintf(msg, sizeof(msg), "Hello from child PID: %d", (int) getpid());
        printf("Child: Child PID: %d\n", (int) getpid());
        write(fd[1], msg, strlen(msg) + 1);
        close(fd[1]);
        exit(0);
    } else {
        /* parent = receiver */
        close(fd[1]); /* close write-end */
        wait(NULL);   /* child prints+sends first -> deterministic output order
     */
        printf("Parent: Parent PID: %d\n", (int) getpid());
        char buf[BUF_SIZE];
        ssize_t n = read(fd[0], buf, sizeof(buf) - 1);
        if (n < 0) {
            fprintf(stderr, "read failed\n");
            exit(1);
        }
        buf[n] = '\0';
        printf("Parent: %s\n", buf);
        close(fd[0]);
        wait(NULL);
    }
    return 0;
}
```

How it works: `pipe(fd)` creates a unidirectional pipe (`fd[0]` = read-end, `fd[1]` = write-end) whose descriptors are shared with the child by `fork()`. Each side closes the end it does not use: the child (sender) writes *Hello from child* + its PID into `fd[1]`, and the parent (receiver) reads it from `fd[0]`. The parent `wait()`s for the child before printing its line, so the output order deterministically matches the expected output.

Results (matches the expected output)

```
Child: Child PID: 2718
Parent: Parent PID: 2717
Parent: Hello from child PID: 2718
```

5 Pipe edge cases (P5)

```c
#include <stdio.h>
#include <stdlib.h>
#include <unistd.h>
#include <string.h>
#include <errno.h>
#include <sys/wait.h>

#define BUF_SIZE 128

/* P5 experiments on pipe behaviour.
   mode 1: sender tries to read back its own message
   mode 2: receiver reads before sender sends (with sleeps to force order)
   mode 3: sender sends several messages before receiver reads */
int main(int argc, char *argv[])
{
    if (argc < 2) {
        fprintf(stderr, "usage: %s <1|2|3>\n", argv[0]);
        exit(1);
    }
    int mode = atoi(argv[1]);
    int fd[2];
    if (pipe(fd) < 0) {
        fprintf(stderr, "pipe failed\n");
        exit(1);
    }
    int rc = fork();
    if (rc < 0) {
        fprintf(stderr, "fork failed\n");
        exit(1);
    } else if (rc == 0) {
        /* child = sender */
        close(fd[0]);
        char msg[BUF_SIZE];
        snprintf(msg, sizeof(msg), "Hello from child PID: %d", (int) getpid());
        printf("Child: Child PID: %d\n", (int) getpid());

        if (mode == 1) {
            /* 5.1 sender tries to read back its own message from its
               read-end (which is closed). */
            char buf[BUF_SIZE];
            ssize_t n = read(fd[0], buf, sizeof(buf));
            if (n < 0)
                printf("Child read() returned %zd (errno=%d: %s)\n",
                       n, errno, strerror(errno));
            else
                printf("Child read() returned %zd (unexpectedly succeeded)\n",
    n);
            write(fd[1], msg, strlen(msg) + 1);
        } else if (mode == 3) {
            /* 5.3 sender sends several messages before receiver reads */
            for (int i = 0; i < 5; i++) {
                char m[BUF_SIZE];
                snprintf(m, sizeof(m), "Message %d from child PID: %d",
                         i + 1, (int) getpid());
                write(fd[1], m, strlen(m) + 1);
                printf("Child sent: %s\n", m);
            }
        } else {
            /* mode 2: parent reads first (handled below) */
            sleep(2); /* guarantee receiver reads before we send */
            printf("Child (sender) sends after delay...\n");
```

```
            write(fd[1], msg, strlen(msg) + 1);
        }
        close(fd[1]);
        exit(0);
    } else {
        /* parent = receiver */
        close(fd[1]);
        if (mode != 2) {
            /* modes 1 and 3: child finishes first -> deterministic output.
               mode 2 must NOT wait: the parent has to read BEFORE the
               sender sends, that is the whole point of the experiment. */
            wait(NULL);
        }
        printf("Parent: Parent PID: %d\n", (int) getpid());
        if (mode == 2) {
            /* 5.2 receiver reads before sender sends: read() should
               block until the sender writes (or EOF if sender closes). */
            printf("Parent: reading (will block until sender writes)...\n");
        }
        char buf[BUF_SIZE * 8];
        ssize_t n;
        while ((n = read(fd[0], buf, sizeof(buf) - 1)) > 0) {
            buf[n] = '\0';
            /* a single read() may return several NUL-separated messages */
            char *p = buf;
            while (p < buf + n) {
                printf("Parent received: %s\n", p);
                p += strlen(p) + 1;
            }
        }
        printf("Parent: read() returned %zd (EOF -> sender closed pipe)\n", n);
        close(fd[0]);
        wait(NULL);
    }
    return 0;
}
```

5.1 Sender tries to read back its own message (5.1)

```
Child: Child PID: 4275
Child read() returned -1 (errno=9: Bad file descriptor)
Parent: Parent PID: 4274
Parent received: Hello from child PID: 4275
Parent: read() returned 0 (EOF -> sender closed pipe)
```

Discussion: The child's `read(fd[0])` returned -1 with `EBADF` because the descriptor was already closed. A closed file descriptor is permanently invalid and cannot be reused.

5.2 Receiver reads before sender sends (5.2)

```
Parent: Parent PID: 4276
Parent: reading (will block until sender writes)...
Child: Child PID: 4277
Child (sender) sends after delay...
Parent received: Hello from child PID: 4277
Parent: read() returned 0 (EOF -> sender closed pipe)
```

Discussion: The parent called `read()` while the pipe was empty (the child slept 2 s before writing). Instead of failing, `read()` blocked until data arrived, then returned the message. A pipe thus acts as a built-in synchronisation mechanism – no data is lost.

5.3 Sender sends several messages before receiver reads (5.3)

```
Child: Child PID: 4284
Child sent: Message 1 from child PID: 4284
Child sent: Message 2 from child PID: 4284
Child sent: Message 3 from child PID: 4284
Child sent: Message 4 from child PID: 4284
Child sent: Message 5 from child PID: 4284
Parent: Parent PID: 4268
Parent received: Message 1 from child PID: 4284
Parent received: Message 2 from child PID: 4284
Parent received: Message 3 from child PID: 4284
Parent received: Message 4 from child PID: 4284
Parent received: Message 5 from child PID: 4284
Parent: read() returned 0 (EOF -> sender closed pipe)
```

Discussion: All five messages were buffered by the kernel and delivered in exact FIFO order with none lost. The pipe's internal buffer (64 KiB by default) allows the sender to write ahead without blocking. In all three experiments, closing the last write-end caused `read()` to return 0 (EOF), which is how the receiver detects that the sender is done.